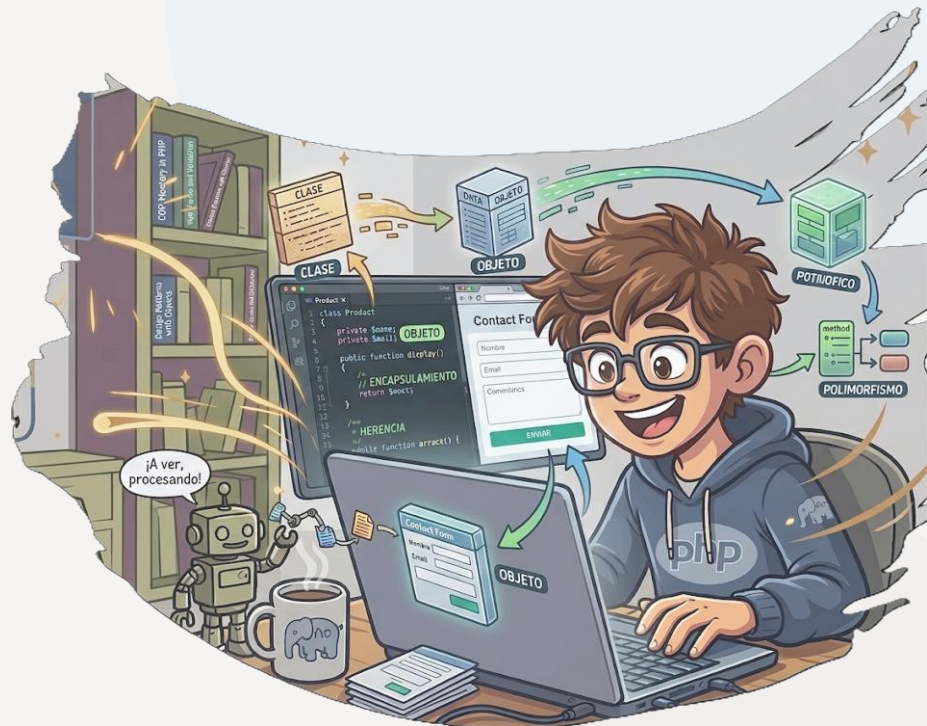


Programación PHP

Conceptos de programación orientada a objetos y manejo de formularios



01

Fundamentos de POO

Fundamentos de la programación orientada a objetos.

02

Manejo de formularios en PHP

Procesamiento, manejo y validación de datos en formularios.

03

Cifrado de Contraseñas

Uso del algoritmo Bcrypt.

04

Gestión de Sesiones

Creación de sesiones con PHP.

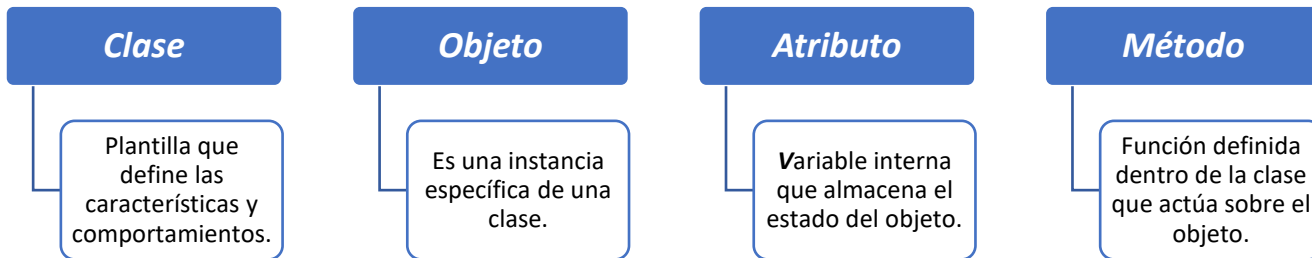
Programación orientada a objetos en PHP

- La **POO** es un paradigma de programación que organiza el código en torno a *objetos* que contienen datos (atributos) y comportamientos (métodos) en una sola entidad.
- PHP soporta POO desde la versión 5.
- La POO promueve la modularidad, reutilización del código, mantenimiento, escalabilidad, abstracción y el polimorfismo.



Programación orientada a objetos en PHP

- Conceptos clave de la programación orientada a objetos:



Programación orientada a objetos en PHP

CLASES

Plantilla que define características y funcionalidades de los objetos.

Tiene atributos (datos) y métodos (comportamiento).



OBJETOS

Es una instancia de una clase.

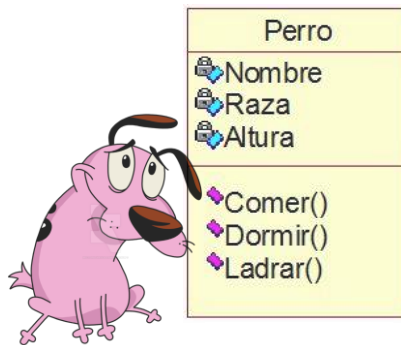
Es la materialización concreta de la plantilla definida por la clase

Es una entidad específica con sus propios datos y comportamiento.



Programación orientada a objetos en PHP

- Un objeto se define como una colección (contenedor) de atributos o propiedades y métodos agrupadas en una estructura.
- **Atributos:** Relación entre un nombre o característica y un valor.
- **Métodos:** Es una función que puede ser llamada.



Concepto de Encapsulación

- Es una estructura de datos que agrupa variables y funciones en una sola unidad denominada **Clase**.
 - Las variables se conocen como *Atributos* o *Propiedades*.
 - Las funciones se conocen como *Métodos*.



Atributos

Contenedores de datos que definen el estado de un objeto. Deben protegerse del acceso externo no autorizado.

Métodos

Funciones que definen el comportamiento. Son la interfaz controlada para interactuar con las propiedades.

Declaración de una clase

- Una clase se declara usando la palabra reservada **class** seguida del nombre de la clase (**por convención, en mayúscula inicial**). Dentro de llaves **{ }** se definen las propiedades y los métodos.



```
<?php
class NombreClase {
    // Atributos
    especificador $nombreAtributo1;
    especificador $nombreAtributo2;
    // Método
    especificador function nombreMetodo()
    {
        //cuerpo del método
        // $this->nombreAtributo
    }
}
```

```
<?php
class Persona {
    // Atributos
    public $nombre;
    public $edad;
    // Método
    public function saludar() {
        echo "Hola, soy $this->nombre";
    }
}
```


Especificadores de acceso a una clase

public



PÚBLICO

Accesible desde
cualquier parte del
código (dentro y fuera
de la clase).

private



PRIVADO

Solo accesible desde
dentro de la misma
clase. No se hereda.

protected



PROTEGIDO

Accesible dentro de
la clase y en clases
hijas (herencia).

Especificadores de acceso a una clase

- Ejemplo:

```
<?php
class CuentaBancaria {
    public $titular;    // Accesible desde fuera
    private $saldo;     // Solo dentro de la clase
    protected $banco;  // Clase + clases hijas

    public function getSaldo() {
        return $this->saldo;
    }
}
```



Especificadores de acceso a una clase

Sintaxis General de Clases en PHP

**NombreDeLaClase**

El nombre de la clase (PascalCase, p.ej., `Usuario`).

**\$propiedad**

Una variable de clase (representa datos).

**function nombreDelMetodo**

Una función de clase (representa comportamiento).

**\$argumento**

Parámetros para el método.

**sentencias**

El código que se ejecuta dentro del método.

```
class NombreDeLaClase
{
    // Propiedades
    public $propiedad1 , $propiedad2 , $propiedad3 ;

    // Métodos
    public function nombreDelMetodo1 ( $argumento1 , $argumento2 , $argumento3 )
    {
        // Sentencias a ejecutar
    }
}
```

public	Accesible desde cualquier lugar.
private	Solo accesible dentro de la misma clase.
protected	Solo accesible dentro de la clase y clases derivadas.

Constructores

- Método especial que se ejecuta en el momento en que se crea un objeto con *new*.
- Se declara siempre con el nombre `__construct`.
- Su propósito es inicializar el objeto; es decir, asignarle valores a sus atributos desde el primer instante.



Constructores

```
<?php
class Producto {
    public $nombre;
    public $precio;
    public $stock;

    // Constructor – se ejecuta con new Producto(...)
    public function __construct($nombre, $precio, $stock = 0) {
        $this->nombre = $nombre;
        $this->precio = $precio;
        $this->stock = $stock;
        echo "Producto creado: " . $this->nombre;
    }

    public function mostrar() {
        echo $this->nombre . " - $" . $this->precio;
    }
}
```



Creación de objetos

- Para crear un objeto se usa la palabra clave **new** seguida del nombre de la clase.



```
<?php
```

```
class Auto {  
    public $marca;  
    public $modelo;  
    public $año;  
}  
  
// Creación de objetos (instancias)  
$auto1 = new Auto();    // Objeto 1  
// Asignando valores a la instancia  
$auto1->marca = "Hyundai";  
$auto1->modelo = "Grand i10";  
$auto1->año = 2026;
```



Acceso a miembros del objeto

- En PHP, se accede a elementos, propiedades y métodos de un objeto utilizando el operador flecha ->.



```
<?php
```

```
class Estudiante {  
    public $nombre;  
    public $nota;  
    public function calificacion() {  
        if ($this->nota >= 60) {return "Aprobado"; }  
        else {return "Reprobado";}  
    }  
    public function mostrarInfo() {  
        echo $this->nombre . ": " . $this->calificacion();  
    }  
}
```



Dentro de la clase: \$this

```
// $this = el objeto actual  
$this->nombre  
$this->nota  
$this->metodo()
```

Fuera de la clase: \$objeto

```
// Con la variable del objeto  
$est->nombre  
$est->nota  
$est->metodo()
```

Funciones require/include

- Insertan el contenido de un archivo PHP dentro de otro.
- Permiten modularizar el código PHP.



Función	Comportamiento ante error	Cuándo usar
<code>include()</code>	Warning · se sigue ejecutando	Archivos opcionales (templates, sidebars)
<code>require()</code>	Fatal error · se detiene todo	Archivos críticos (clases, config, BD)
<code>include_once()</code>	Warning · se incluye solo 1 vez	Evitar doble inclusión de archivos opcionales
<code>require_once()</code>	Fatal error · se incluye solo 1 vez	Clases/librerías (evita redeclaración)

Funciones require/include

```
<?php
```

```
// archivo: Persona.php (define la clase)
```

```
class Persona { ... }
```

```
// archivo: index.php (usa la clase)
```

```
require_once("Persona.php"); // clase incluida una sola vez
```

```
include("encabezado.php"); // template (no es crítico)
```

```
$p = new Persona("Carlos", 30);
```

```
$p->saludar();
```



Formularios HTML

Etiqueta `<input>` `</input>`

- Permite crear controles o componentes interactivos para formularios basados en la web.
- Dispone de una amplia variedad de datos de entrada.
- Atributos principales
 - ***type***: define el tipo de formulario de entrada que se está utilizando.
 - ***value***: Define un valor inicial para el control.
 - ***placeholder***: Ilustra al usuario para que éste sepa que debe escribir.
 - ***autofocus***: Coloca el cursor en un campo al cargar la página.



Formularios HTML

<code><input type="text"></code>	xyz
<code><input type="password"></code>	*****
<code><input type="radio"></code>	No ☐ Yes ☒
<code><input type="checkbox"></code>	☐ ☐ ☒
<code><input type="button"></code>	Button
<code><input type="color"></code>	<input type="color" value="#0000FF"/>
<code><input type="email"></code>	<input type="email" value="xyz@gmail.com"/>
<code><input type="file"></code>	<input type="file" value="Choose File image.jpg"/>
<code><input type="hidden"></code>	<input type="hidden"/>
<code><input type="image"></code>	<input type="image" value="Submit Image"/>
<code><input type="number"></code>	<input type="number" value="898"/>



Formularios HTML

- Un formulario `<form>` `</form>` permite enviar datos hacia el servidor web, para ser procesados mediante el uso de un lenguaje script.
- Es un contenedor de controles interactivos que se utiliza para recolectar diferentes tipos de datos: `<input>`, `<label>`, `<button>`, etc.
- Sintaxis:

```
<form method="POST" action="datos.php">
```

[Conjunto de controles de formulario]

```
</form>
```



Formularios HTML

Atributos <form> </form>

- **method:** Indica el método HTTP de envío de los datos.
 - **GET:** Los datos se envían como parte de la URL, es visible en la barra de direcciones. Se añaden con un “?”.
 - **POST:** Envía en segundo plano, de forma oculta. Los datos se envían en el cuerpo de la solicitud HTTP.
- **action:** Define ubicación donde se envía los datos para ser procesados (script).
 - **Ejemplos:** datos.php, datos.py



Manejo de datos en formularios

Uso de variables Superglobales

- Las variables superglobales capturan automáticamente los datos enviados desde formularios HTML.

\$_POST	\$_GET
Recibe datos enviados por método POST. No son visibles en la URL.	Recibe datos enviados por método GET. Visibles en la URL del navegador.
\$_SERVER	\$_REQUEST
Información del servidor y la solicitud HTTP (IP cliente, método, ruta del script, etc.).	Combina \$_POST, \$_GET y \$_COOKIE en un solo arreglo.

Validación y Sanitización de Datos

- Nunca se debe confiar en los datos del usuario. Todo dato de un formulario debe validarse y sanitizarse antes de usarse o guardarse.
- Validar, verificar que el dato sea correcto:

```
<?php
$email = $_POST["email"];
if (empty($email)) {
    echo "El email es obligatorio";
}
if (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
    echo "Email no válido";
}
?>
```

empty()

Verifica si la variable está vacía.

filter_var()

Valida/sanitiza con filtros de PHP.

Validación y Sanitización de Datos

- Sanitizar, se limpia el dato recibido antes de utilizarlo:

```
<?php
    $nombre = $_POST["nombre"];

    $nombre = strip_tags($nombre);    // quita HTML
    $nombre = trim($nombre);          // quita espacios
    $nombre = htmlspecialchars($nombre);
    echo $nombre; // dato limpio
?>
```

strip_tags()

Elimina etiquetas HTML y PHP.

trim()

Elimina espacios al inicio y final.

htmlspecialchars()

Convierte < > & " en entidades HTML.

intval()

Convierte el valor a número entero.



Ataques XSS e Inyección SQL

Validar en el servidor

Validar siempre en el servidor, las validaciones en el cliente (JS) se pueden evadir.

htmlspecialchars()

Se recomienda su uso para prevenir ataque XSS.
(Cross-Site Scripting)

Sentencias preparadas

Usar consultas parametrizadas para evitar inyección SQL.

Uso de hash

Almacenar contraseñas con hash.

La validación y sanitización es la primera línea de defensa, útil para rechazar datos claramente inválidos y dar buena experiencia de usuario.



Cifrado de Contraseñas

- PHP ofrece funciones nativas para cifrar y verificar contraseñas de forma segura.
- Se utiliza el algoritmo Bcrypt y se implementa a través de las funciones:

password_hash()

```
<?php
$clave = $_POST["password"];
$hash = password_hash($clave, PASSWORD_BCRYPT);
// Guardar $hash en la BD, nunca $clave directa
?>
```

password_verify()

```
<?php
$clave = $_POST["password"];
$hashBD = /* hash guardado en la BD */;
if (password_verify($clave, $hashBD)) {
    echo "Acceso permitido";
} else {echo "Contraseña incorrecta"; }
?>
```

Gestión de sesiones

- Las sesiones pueden definirse como una serie de variables almacenadas en el servidor, asociadas a un usuario mediante un identificador único (session ID), que permiten mantener información de manera persistente entre distintas peticiones HTTP durante su visita a la aplicación.
- `session_start()` es necesario en cualquier página que use `$_SESSION`. Buena práctica: centralizarlo en un archivo incluido al principio de cada página.

Sintaxis clave



`session_start()`: genera un identificador único de sesión, envía ese ID al navegador como cookie y carga los datos previos si la sesión ya existía.

`$_SESSION[]`: variable superglobal; arreglo asociativo que guarda los datos de sesión actual del usuario.

`session_unset()`: borra las variables, pero la sesión sigue active.

`session_destroy()`: elimina por completo los datos de sesión en el servidor.

Manejo de Excepciones

- Una excepción es un error o condición especial que ocurre durante la ejecución de un programa y que interrumpe el flujo normal de instrucciones.
- En PHP las excepciones se gestionan mediante el bloque: **try-throw-catch**.
- En formularios web, sirve para detectar campos vacíos o inválidos y evitar que datos corruptos lleguen a la base de datos.



Manejo de Excepciones

- **Sintaxis básica:**

```
<?php
try {
    // Código que puede fallar
    if (empty($_POST['cedula'])) {
        throw new Exception("Cédula es un campo obligatorio"); }
    echo "Datos válidos";
} catch (Exception $e) {
    echo "Error: " . $e->getMessage(); // Captura y maneja el error
} finally {
    // Se ejecuta siempre (opcional)
    echo "Validación finalizada";
}
?>
```

Puntos clave

- **throw new Exception():** crea y lanza el objeto de error.
- **\$e->getMessage():** obtiene el texto del error.
- **catch (Exception \$e):** captura cualquier excepción genérica.
- **finally:** bloque opcional que siempre se ejecuta.



CURSO DE PROGRAMACIÓN VII.

Licenciatura en Desarrollo y Gestión de Software. FISC 2026.